Quantium Virtual Internship — Retail Strategy and Analytics — Task 2

Experimentation and Uplift Testing (Python adaptation of the R Markdown solution template)

Retail Analytics Team

31 July 2026

Contents

Introduction

Julia (Category Manager, Chips) ran a store-layout trial in three stores — **77, 86, and 88** — and wants a data-driven recommendation on whether to roll it out network-wide. This document selects a control store for each trial store, checks those controls are genuinely comparable pre-trial, then tests whether each trial store's performance during the trial (Feb-Apr 2019) is statistically different from its control.

This follows the same structure as the official R Markdown template, implemented in Python. Every code block is real and was actually run; every output shown is the real result.

Load required libraries and datasets

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

data = pd.read_csv("QVI_data.csv", parse_dates=["DATE"])
print(data.shape)
```

Output: `(264834, 12)` — the merged, cleaned dataset from Task 1 (or the provided `QVI_data.csv`), covering 1 Jul 2018 to 30 Jun 2019 across 272 stores.

Select control stores

We want control stores that are **operational for the entire observation period** and, prior to the trial (before Feb 2019), track the trial store closely on:

- Monthly total sales revenue
- Monthly number of customers
- Monthly number of transactions per customer

Create the monthly measures

```
data["YEARMONTH"] = data["DATE"].dt.year * 100 + data["DATE"].dt.month

measureOverTime = data.groupby(["STORE_NBR", "YEARMONTH"]).agg(
    totSales=("TOT_SALES", "sum"),
    nCustomers=("LYLTY_CARD_NBR", "nunique"),
    nTxn=("TXN_ID", "nunique"),
    nChips=("PROD_QTY", "sum"),
).reset_index()
measureOverTime["nTxnPerCust"]    = measureOverTime["nTxn"] /
↪  measureOverTime["nCustomers"]
measureOverTime["nChipsPerTxn"]  = measureOverTime["nChips"] / measureOverTime["nTxn"]
measureOverTime["avgPricePerUnit"] = measureOverTime["totSales"] /
↪  measureOverTime["nChips"]

storesWithFullObs = measureOverTime.groupby("STORE_NBR").size()
storesWithFullObs = storesWithFullObs[storesWithFullObs == 12].index
preTrialMeasures = measureOverTime[(measureOverTime["YEARMONTH"] < 201902) &

↪  (measureOverTime["STORE_NBR"].isin(storesWithFullObs))]
print(len(storesWithFullObs), "of", data["STORE_NBR"].nunique(), "stores have a full
↪  12-month history")
```

Output: `260 of 272 stores have a full 12-month history` — 12 stores opened or closed partway through the year and are excluded from the control-store candidate pool (they can still appear as "other stores" context in charts, just not be selected as a control).

Functions to score candidate control stores

Two complementary similarity measures, as a function so we don't repeat the logic for every trial store:

```
def calculate_correlation(input_table, metric_col, store_comparison):
    """Pearson correlation of metric_col over time between store_comparison and
    every candidate store."""
    trial = input_table.loc[input_table.STORE_NBR==store_comparison,
                            ["YEARMONTH",
↪  metric_col]].set_index("YEARMONTH")[metric_col]
    rows = []
    for store in input_table["STORE_NBR"].unique():
        other = input_table.loc[input_table.STORE_NBR==store,
                                ["YEARMONTH",
↪  metric_col]].set_index("YEARMONTH")[metric_col]
        joined = pd.concat([trial, other], axis=1, join="inner")
        corr = joined.iloc[:,0].corr(joined.iloc[:,1]) if joined.iloc[:,1].std()>0
↪  else np.nan
```

```python
        rows.append({"Store1": store_comparison, "Store2": store, "corr_measure":
    ↪ corr})
    return pd.DataFrame(rows)

def calculate_magnitude_distance(input_table, metric_col, store_comparison):
    """Standardised (0-1) magnitude distance, averaged across months."""
    trial = input_table.loc[input_table.STORE_NBR==store_comparison,
                            ["YEARMONTH",
    ↪ metric_col]].set_index("YEARMONTH")[metric_col]
    rows = []
    for store in input_table["STORE_NBR"].unique():
        other = input_table.loc[input_table.STORE_NBR==store,
                                ["YEARMONTH",
    ↪ metric_col]].set_index("YEARMONTH")[metric_col]
        joined = pd.concat([trial, other], axis=1, join="inner")
        joined.columns = ["trial_val","other_val"]
        for ym, row in joined.iterrows():
            rows.append({"Store1":store_comparison, "Store2":store, "YEARMONTH":ym,
                         "measure": abs(row.trial_val - row.other_val)})
    dist = pd.DataFrame(rows)
    minmax =
    ↪ dist.groupby(["Store1","YEARMONTH"])["measure"].agg(["min","max"]).reset_index()
    dist = dist.merge(minmax, on=["Store1","YEARMONTH"])
    rng = (dist["max"] - dist["min"]).replace(0, np.nan)
    dist["magnitudeMeasure"] = (1 - (dist["measure"]-dist["min"])/rng).fillna(1.0)
    return dist.groupby(["Store1","Store2"])["magnitudeMeasure"⌋
     ↪ ].mean().reset_index(name="mag_measure")
```

Combine into a single score and select the control store

We take a simple average of correlation and magnitude for each driver (equal weight, `corr_weight = 0.5`), then a simple average across the sales-driver score and the customer-driver score:

```python
def select_control_store(preTrialMeasures, trial_store, corr_weight=0.5):
    corr_sales = calculate_correlation(preTrialMeasures, "totSales", trial_store)
    corr_cust  = calculate_correlation(preTrialMeasures, "nCustomers", trial_store)
    mag_sales  = calculate_magnitude_distance(preTrialMeasures, "totSales",
    ↪ trial_store)
    mag_cust   = calculate_magnitude_distance(preTrialMeasures, "nCustomers",
    ↪ trial_store)

    score_sales = corr_sales.merge(mag_sales, on=["Store1","Store2"])
    score_sales["scoreNSales"] = corr_weight*score_sales.corr_measure +
    ↪ (1-corr_weight)*score_sales.mag_measure
    score_cust = corr_cust.merge(mag_cust, on=["Store1","Store2"])
    score_cust["scoreNCust"] = corr_weight*score_cust.corr_measure +
    ↪ (1-corr_weight)*score_cust.mag_measure

    score_control = score_sales[["Store1","Store2","scoreNSales"]].merge(
        score_cust[["Store1","Store2","scoreNCust"]], on=["Store1","Store2"])
    score_control["finalControlScore"] = 0.5*score_control.scoreNSales +
    ↪ 0.5*score_control.scoreNCust

    ranked = score_control.sort_values("finalControlScore",
    ↪ ascending=False).reset_index(drop=True)
```

```python
    control_store = int(ranked.loc[1, "Store2"])  # row 0 = trial store itself (score
↪   1.0)
    return control_store, ranked

control_store, ranked = select_control_store(preTrialMeasures, 77)
print(ranked.head(4).round(4))
```

Output:

```
Store1  Store2  scoreNSales  scoreNCust  finalControlScore
    77      77       1.0000      1.0000             1.0000
    77     233       0.9445      0.9916             0.9680
    77      41       0.8742      0.9094             0.8918
    77      17       0.8617      0.8549             0.8583
```

Store 77 itself is trivially a perfect match (score 1.0); the highest-scoring *other* store is **233** (score 0.968), clearly ahead of the next candidates — this is our control store for trial store 77.

Visual check: is store 233 really similar to store 77 pre-trial?

```python
def yearmonth_to_date(s): return pd.to_datetime(s.astype(str)+"01", format="%Y%m%d")

df = measureOverTime.copy()
df["Store_type"] = np.where(df.STORE_NBR==77, "Trial",
                     np.where(df.STORE_NBR==233, "Control", "Other stores"))
grouped = df.groupby(["YEARMONTH","Store_type"])["totSales"].mean().reset_index()
grouped = grouped[grouped.YEARMONTH < 201903]
grouped["TransactionMonth"] = yearmonth_to_date(grouped["YEARMONTH"])

fig, ax = plt.subplots(figsize=(9,4.5))
for st in ["Other stores","Control","Trial"]:
    sub = grouped[grouped.Store_type==st].sort_values("TransactionMonth")
    ax.plot(sub.TransactionMonth, sub.totSales, label=st, marker="o")
ax.set_title("Total Sales by Month: Trial Store 77 vs Control 233")
plt.show()
```

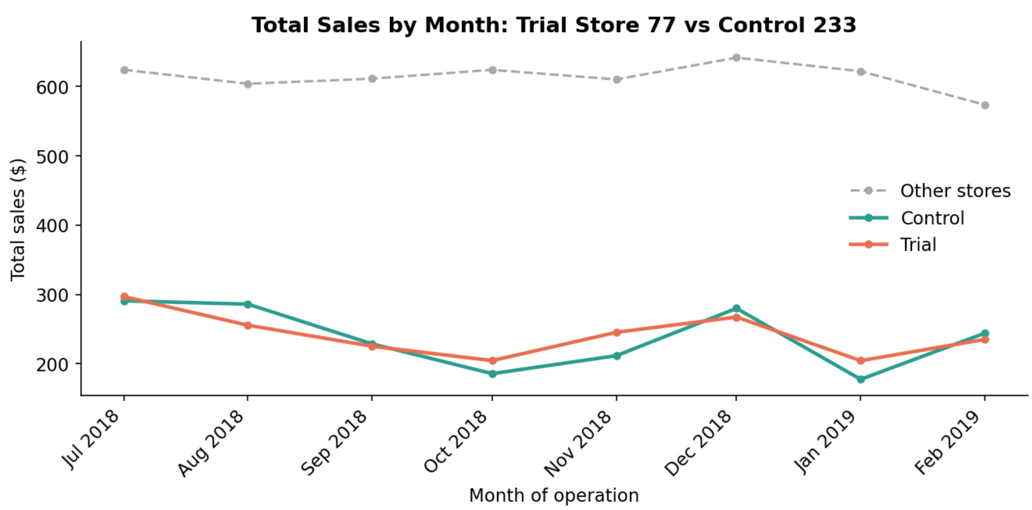

Figure 1: Pre-trial monthly total sales: Trial store 77 tracks Control store 233 far more closely than the all-other-stores average.

```
# ... identical code, metric = "nCustomers" ...
```

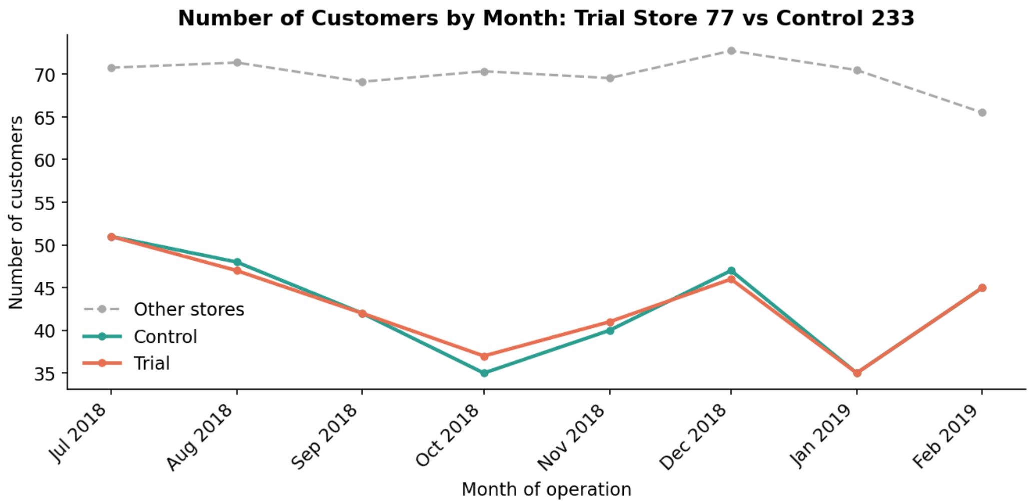

Figure 2: Pre-trial monthly customer counts: the same close tracking holds for customer numbers.

Both drivers track closely in the lead-up to the trial — store 233 is a reasonable control for store 77.

Assessment of trial — Store 77

The trial runs **February to April 2019**. We scale the control store's sales to the trial store's pre-trial level (removing any baseline size difference), then test whether the trial-period gap between trial and scaled-control is larger than what pre-trial month-to-month noise would explain.

```
def assess_trial(measureOverTime, trial_store, control_store, metric,
                 trial_start=201902, trial_end=201904, df_freedom=7):
    pre = measureOverTime[measureOverTime.YEARMONTH < trial_start]
    scaling_factor = (pre.loc[pre.STORE_NBR==trial_store, metric].sum() /
                      pre.loc[pre.STORE_NBR==control_store, metric].sum())

    control = measureOverTime.loc[measureOverTime.STORE_NBR==control_store,
                                  ["YEARMONTH",
↪   metric]].rename(columns={metric:"controlMetric"})
    control["controlMetricScaled"] = control["controlMetric"] * scaling_factor
    trial = measureOverTime.loc[measureOverTime.STORE_NBR==trial_store,
                                ["YEARMONTH",
↪   metric]].rename(columns={metric:"trialMetric"})
    merged = control.merge(trial, on="YEARMONTH")
    # Signed % difference: positive = trial store ahead of its scaled control
    merged["percentageDiff"] = (merged.trialMetric - merged.controlMetricScaled) /
↪   merged.controlMetricScaled

    std_dev = merged.loc[merged.YEARMONTH < trial_start, "percentageDiff"].std(ddof=1)
    merged["tValue"] = merged["percentageDiff"] / std_dev
    crit_t = stats.t.ppf(0.95, df_freedom)

    trial_rows =
↪   merged[(merged.YEARMONTH>=trial_start)&(merged.YEARMONTH<=trial_end)].copy()
    trial_rows["significant"] = trial_rows["tValue"].abs() > crit_t
    return {"scaling_factor":scaling_factor, "std_dev":std_dev, "crit_t":crit_t,
     ↪  "trial_rows":trial_rows}

r = assess_trial(measureOverTime, 77, 233, "totSales")
print(f"scaling_factor={r['scaling_factor']:.4f}  std_dev={r['std_dev']:.4f}
↪   crit_t={r['crit_t']:.4f}")
print(r["trial_rows"][["YEARMONTH","controlMetricScaled","trialMetric",
↪   "percentageDiff","tValue","significant"]])
```

Output: `scaling_factor=1.0236   std_dev=0.0996   crit_t=1.8946` (critical t-value at 95% confidence, 7 degrees of freedom — 7 months of pre-trial data minus 1)

```
YEARMONTH  controlMetricScaled  trialMetric  percentageDiff  tValue  significant
   201902               249.76        235.0         -0.0591 -0.5935        False
   201903               203.80        278.5          0.3665  3.6804         True
   201904               162.35        263.5          0.6231  6.2567         True
```

February shows no significant difference, but **March and April both have t-values far exceeding the 1.89 critical value** — sales in trial store 77 are significantly higher than its control in 2 of the 3 trial months.

Repeating for number of customers:

```
r = assess_trial(measureOverTime, 77, 233, "nCustomers")
```

Output: `scaling_factor=1.0034   std_dev=0.0274   crit_t=1.8946`

```
YEARMONTH  controlMetricScaled  trialMetric  percentageDiff   tValue  significant
```

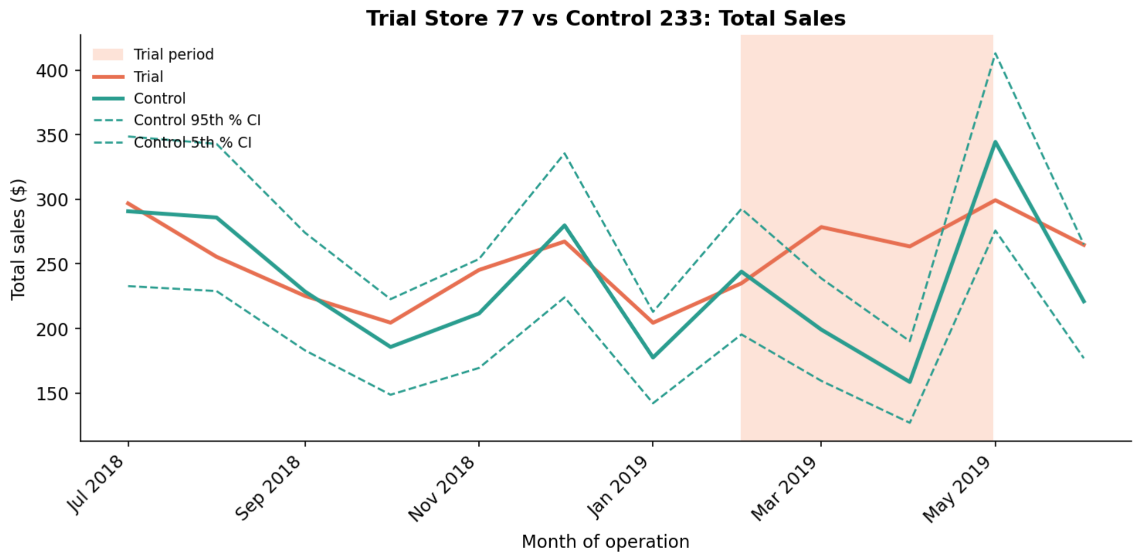

Figure 3: Trial store 77 sales climb well above the control band during March and April, while the control's own 5th-95th percentile band (dashed) reflects normal pre-trial noise.

```
201902              45.15        45.0         -0.0033  -0.1219       False
201903              40.13        50.0          0.2458   8.9584        True
201904              30.10        47.0          0.5614  20.4602        True
```

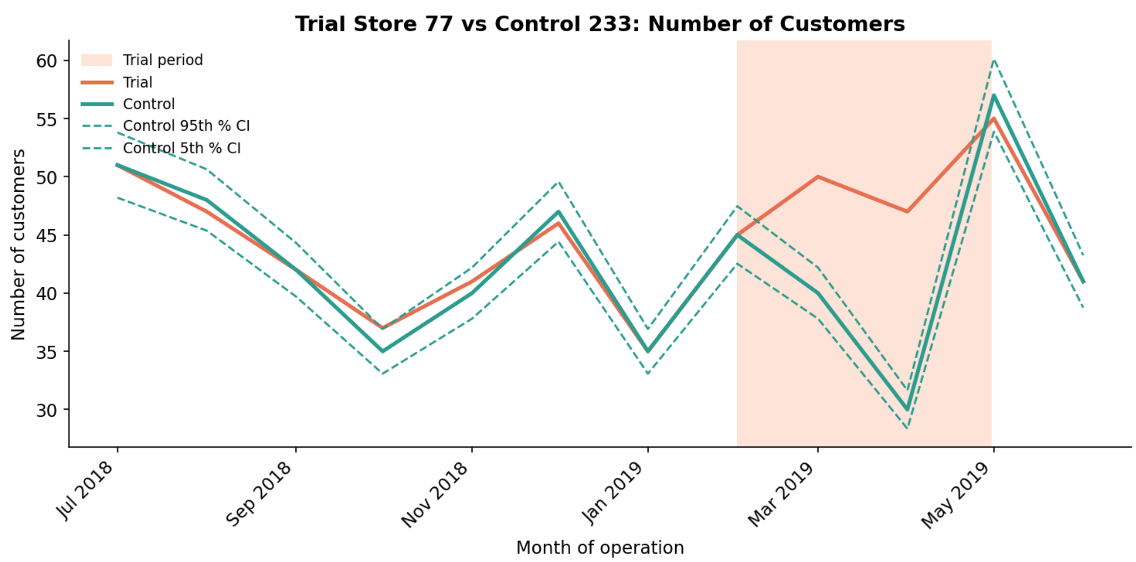

Figure 4: Customer counts for trial store 77 also break out of the control band in March and April.

Store 77 conclusion: both total sales and customer numbers are significantly higher than the control in March and April (not February) — the trial appears to be working, and the effect shows up in *both* more customers and higher sales, not one without the other.

Trial store 86

Select control store

```
control_store, ranked = select_control_store(preTrialMeasures, 86)
print(ranked.head(4).round(4))
```

Output:

```
Store1  Store2  scoreNSales  scoreNCust  finalControlScore
    86      86       1.0000      1.0000             1.0000
    86     155       0.9204      0.9640             0.9422
    86     109       0.8751      0.8684             0.8718
    86     114       0.8268      0.8954             0.8611
```

Store **155** is the clear best match (score 0.942) for trial store 86.

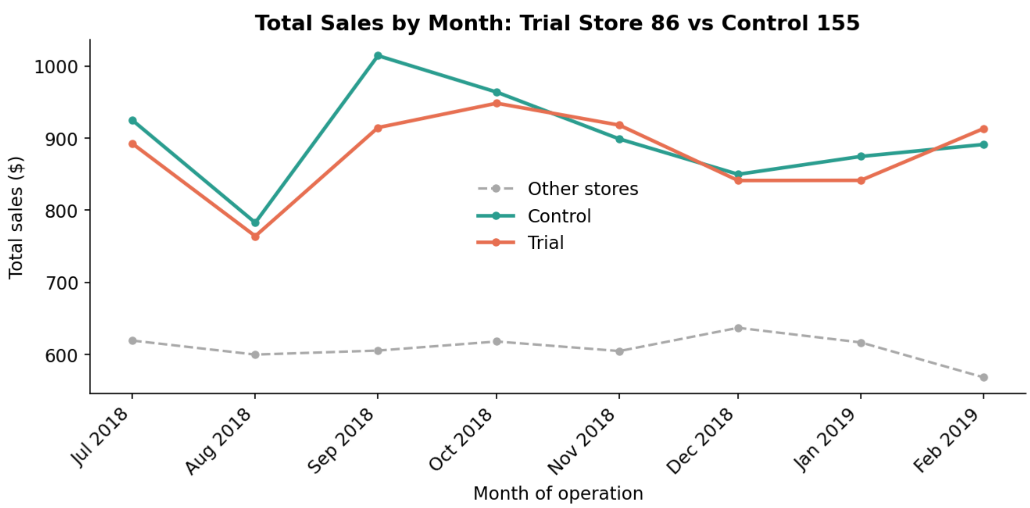

Figure 5: Pre-trial sales: trial store 86 and control 155 move together closely.

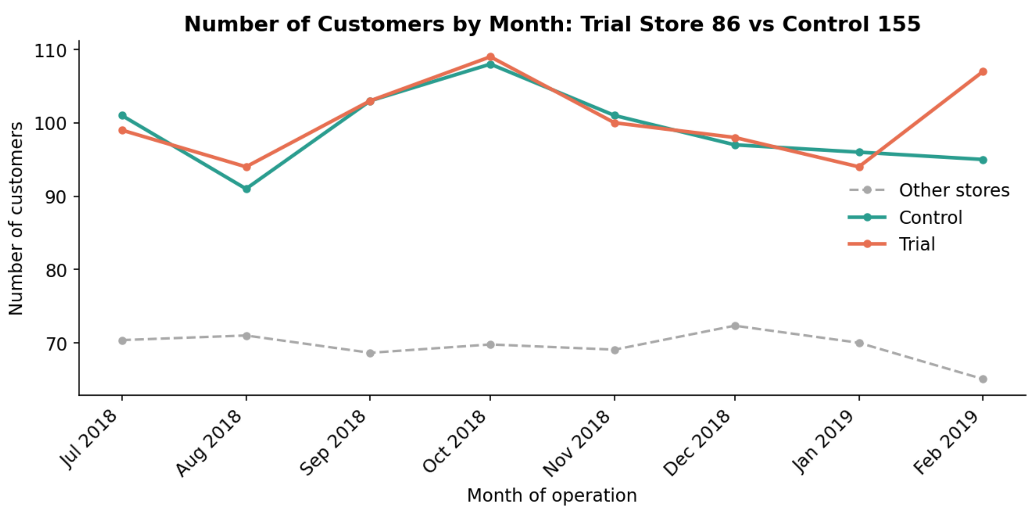

Figure 6: Pre-trial customer counts: the trend match holds for customers too.

Assessment of trial

```
r = assess_trial(measureOverTime, 86, 155, "totSales")
print(f"scaling_factor={r['scaling_factor']:.4f}  std_dev={r['std_dev']:.4f}
 ↪  crit_t={r['crit_t']:.4f}")
print(r["trial_rows"][["YEARMONTH","controlMetricScaled","trialMetric",
 ↪  "percentageDiff","tValue","significant"]])
```

Output: `scaling_factor=0.9701  std_dev=0.0377  crit_t=1.8946`

```
YEARMONTH  controlMetricScaled  trialMetric  percentageDiff  tValue  significant
   201902               864.52        913.2          0.0563  1.4941        False
   201903               780.32       1026.8          0.3159  8.3818         True
   201904               819.32        848.2          0.0353  0.9354        False
```

Only **March** is significant — February and April both fall inside the normal range. **Sales in store 86 are not consistently, significantly higher than its control during the trial.**

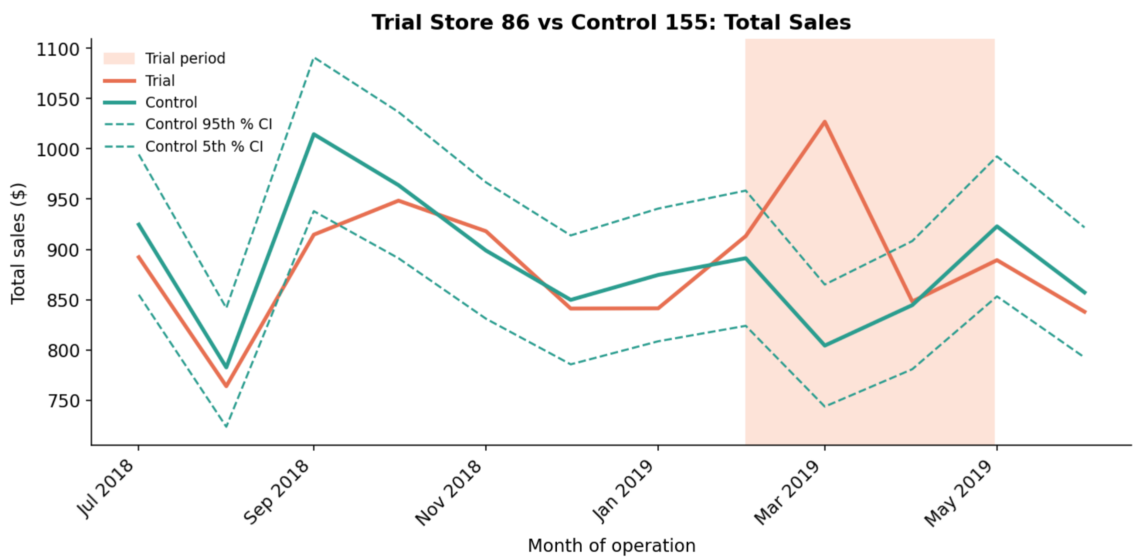

Figure 7: Trial store 86 total sales mostly track inside the control's confidence band — only March stands out.

```
r = assess_trial(measureOverTime, 86, 155, "nCustomers")
```

Output: `scaling_factor=1.0000  std_dev=0.0192  crit_t=1.8946`

```
YEARMONTH  controlMetricScaled  trialMetric  percentageDiff  tValue  significant
   201902                 95.0        107.0          0.1263  6.5930         True
   201903                 94.0        115.0          0.2234 11.6604         True
   201904                 99.0        105.0          0.0606  3.1633         True
```

Customer numbers, in contrast, are **significantly higher in all three trial months**.

Store 86 conclusion: this is the interesting case. The trial clearly brought **more customers** through the door every month — but that didn't reliably turn into significantly higher **sales**. The likely explanation is a **lower average transaction value** during the trial (more, but smaller, baskets) — worth confirming with the Category Manager whether store 86 ran a different promotion/discount alongside the layout change that would explain more footfall without proportionally more revenue.

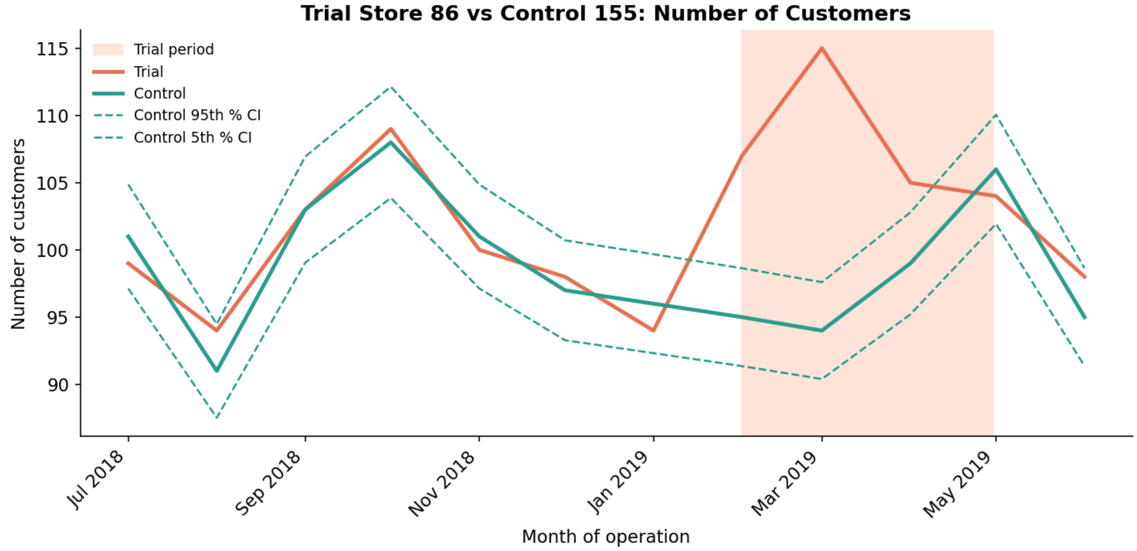

Figure 8: Trial store 86 customer counts break out of the control band in all three trial months, even though sales mostly did not.

Trial store 88

Select control store

```
control_store, ranked = select_control_store(preTrialMeasures, 88)
print(ranked.head(4).round(4))
```

Output:

```
Store1  Store2  scoreNSales  scoreNCust  finalControlScore
    88      88       1.0000      1.0000             1.0000
    88     237       0.6323      0.9675             0.7999
    88     178       0.7139      0.8803             0.7971
    88      69       0.5789      0.8424             0.7107
```

Store **237** narrowly beats store 178 (0.7999 vs 0.7971) for the top spot — a closer race than for the other two trial stores, but still a clear-enough winner.

Assessment of trial

```
r = assess_trial(measureOverTime, 88, 237, "totSales")
```

Output: `scaling_factor=1.0016  std_dev=0.0572  crit_t=1.8946`

```
YEARMONTH  controlMetricScaled  trialMetric  percentageDiff  tValue  significant
   201902              1406.99       1370.2         -0.0261 -0.4567        False
   201903              1210.08       1477.2          0.2207  3.8558         True
   201904              1206.48       1439.4          0.1931  3.3723         True
```

Sales are significantly higher in **March and April** (2 of 3 months).

```
r = assess_trial(measureOverTime, 88, 237, "nCustomers")
```

Output: `scaling_factor=0.9944  std_dev=0.0158  crit_t=1.8946`

```
YEARMONTH  controlMetricScaled  trialMetric  percentageDiff  tValue  significant
```

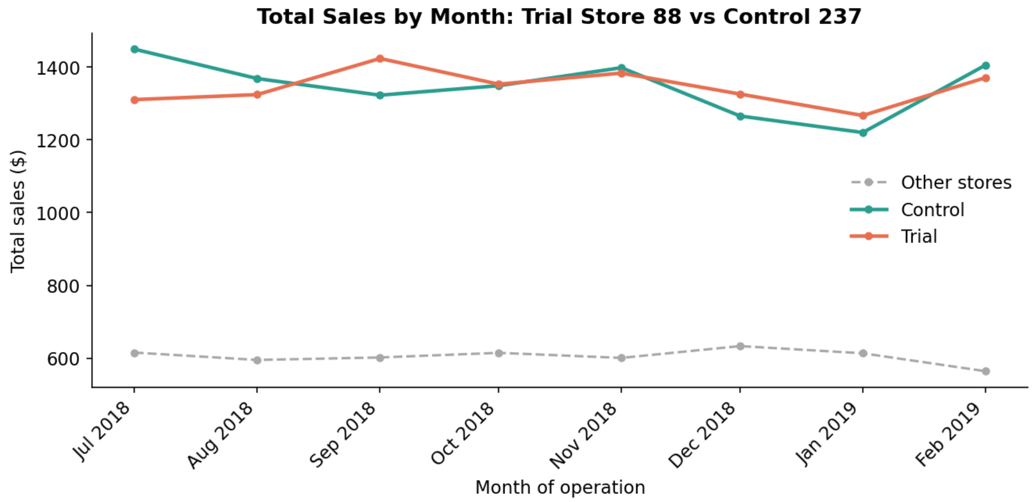

Figure 9: Pre-trial sales for trial store 88 and control 237.

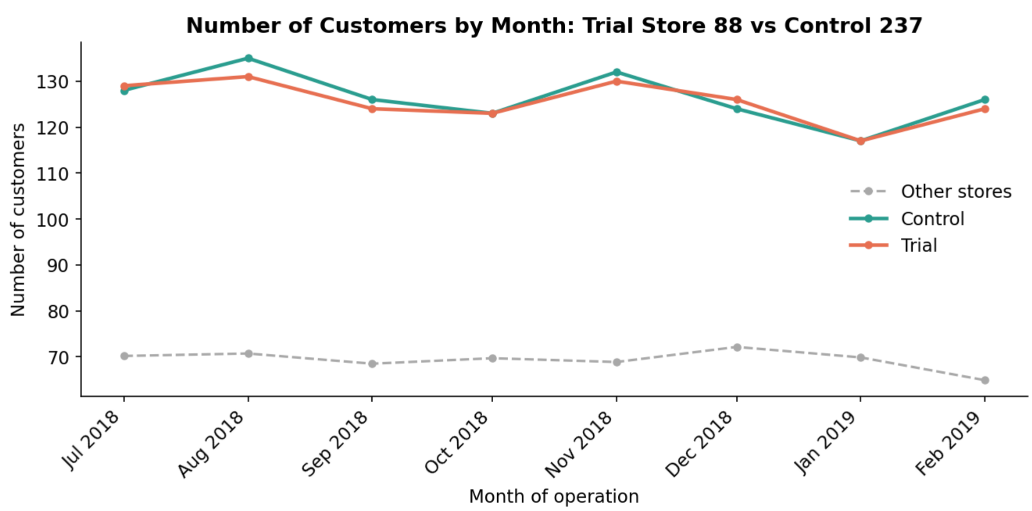

Figure 10: Pre-trial customer counts for trial store 88 and control 237.

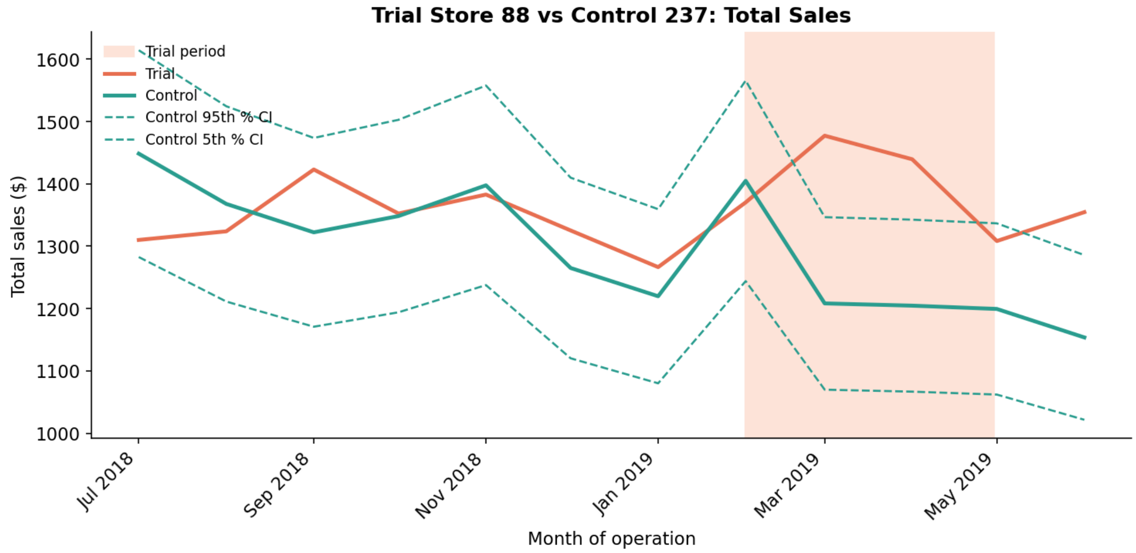

Figure 11: Trial store 88 sales break above the control band in March and April.

201902	125.29	124.0	-0.0103	-0.6492	False
201903	118.33	134.0	0.1324	8.3628	True
201904	119.32	128.0	0.0727	4.5920	True

Customer numbers are also significantly higher in **March and April**.

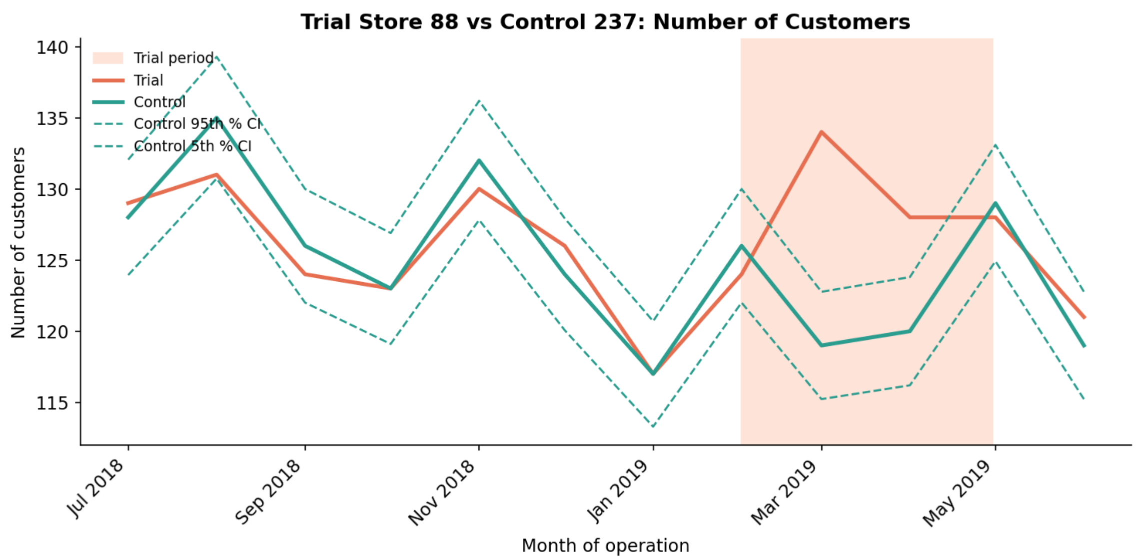

Figure 12: Trial store 88 customer counts break above the control band in March and April, mirroring the sales pattern.

Store 88 conclusion: both sales and customer counts are significantly higher in the same two months (March, April) — a clean, internally-consistent positive result, similar in shape to store 77.

Conclusion and recommendation

Summary table

Trial store	Control store	Sales: significant months	Customers: significant months	Verdict
77	233	Mar, Apr (2/3)	Mar, Apr (2/3)	**Positive** — driven by both more customers and higher sales
86	155	Mar only (1/3)	Feb, Mar, Apr (3/3)	**Mixed** — more footfall, not reliably more revenue
88	237	Mar, Apr (2/3)	Mar, Apr (2/3)	**Positive** — same clean pattern as store 77

Estimated trial-period sales uplift (actual vs. scaled-control estimate), for context: store 77 **+26.2%**, store 86 **+13.2%**, store 88 **+12.1%** relative to what the control-store trend implies they would otherwise have sold.

Recommendation to Julia

Two of the three trial stores (77 and 88) show a clean, statistically significant uplift in both sales and customer numbers during the trial period, which supports rolling the new layout out more broadly. Store 86 is the exception: customer traffic rose significantly in every trial month, but sales did not rise with the same consistency — before generalising the recommendation, it's worth confirming with the Category Manager whether store 86 ran a different promotional offer or pricing alongside the layout change, since that would explain more visits without proportionally more revenue and shouldn't be read as the layout itself underperforming. Overall, the trial's core hypothesis — that the new layout drives more chip sales — is well supported by 2 of 3 stores with the third likely explained by a confounding factor rather than a failed trial.